



Manual de Despliegue en la Nube

API de Readmisión Hospitalaria contenerizada con Docker

Materia:	Gestión de Proyectos de Inteligencia Artificial
Actividad:	Fase II — Solución técnica desplegable
Alumno(a):	JUAN ANDRES GUTIERREZ MARTINEZ
Matrícula:	07144441
Docente:	LUIS ARIEL VAZQUEZ PEÑA
Fecha de entrega:	12 / 07 / 2026

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Se completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	☐ No	
Claude (Anthropic)	☒ Sí	Generación y documentación del código, redacción técnica del manual, Dockerfile y estrategia de despliegue.
Gemini (Google)	☐ No	
Copilot (Microsoft)	☐ No	
Otra (especifica):	☐ No	

Desarrollo de Contenido

1. Introducción

Un modelo de aprendizaje automático que ofrece un desempeño sobresaliente en el entorno de desarrollo no garantiza, por sí mismo, resultados consistentes al trasladarse a otros entornos. Las diferencias en dependencias, versiones de librerías y configuraciones del sistema operativo suelen provocar inconsistencias en las predicciones. Este documento presenta el manual de despliegue de una solución técnica completa y desplegable que resuelve dicho problema mediante la contenerización con Docker.

La solución encapsula un modelo de clasificación —predicción de readmisión hospitalaria a 30 días en pacientes con enfermedades crónicas— junto con su código, dependencias y entorno de ejecución, en una unidad portable, aislada y reproducible. El modelo se expone como una API REST (Flask) integrada con un frontend web, y se prepara su publicación en la nube bajo un modelo de servicio PaaS.

Objetivo

Desarrollar y documentar una solución técnica completa y desplegable que garantice portabilidad, consistencia y reproducibilidad del modelo desde el entorno local hasta la nube.

Alcance

- Contenerización de la aplicación (modelo + API + frontend) mediante Docker.
- Validación del funcionamiento en un entorno local comprobable.
- Integración funcional entre frontend y backend a través de una API REST.
- Estrategia y procedimiento paso a paso para el despliegue en la nube (PaaS).
- Documentación técnica alineada con la norma ISO/IEC 23053 y apoyada en herramientas de IA generativa.

2. Desarrollo

2.1 Descripción de la solución y arquitectura

La solución sigue una arquitectura cliente-servidor de tres capas. El frontend (interfaz web) captura las variables clínicas del paciente y las envía mediante HTTP/JSON al backend. El backend (API REST en Flask) recibe la solicitud, valida los datos, carga el modelo serializado (model.pkl) y devuelve la predicción con su probabilidad. El modelo aplica un umbral operativo de 0.30 para priorizar el recall (no omitir pacientes de alto riesgo).

Frontend (HTML/JS) → API REST (Flask, app.py) → Modelo (Random Forest, model.pkl)

Componente	Archivo	Responsabilidad
Entrenamiento offline	entrenar_modelo.py	Entrena el Random Forest y serializa model.pkl
Backend / API	app.py	Expone los endpoints REST y sirve el frontend

Componente	Archivo	Responsabilidad
Frontend	templates/index.html	Formulario web que consume POST /predict
Simulación local	simular.py	Casos comunes, extremos y validación con probabilidades
Cliente de prueba	cliente_api.py	Consume la API por HTTP (evidencia de integración)
Contenedor	Dockerfile / docker-compose.yml	Definición de la imagen e instrucciones de ejecución

2.2 Requerimientos técnicos

Categoría	Requerimiento
Sistema operativo	Windows, Linux o macOS con Docker Engine 24+ (probado en 29.6.1)
Lenguaje	Python 3.11
Contenerización	Docker / Docker Compose
Framework web	Flask 3.1 (servido con gunicorn en producción)
Librerías de ML	scikit-learn 1.9, joblib 1.5, numpy 2.2, pandas 2.2
Control de versiones	Git + repositorio en GitHub
Puerto expuesto	5000/TCP

Las versiones se fijan en requirements.txt para asegurar reproducibilidad entre entornos, evitando las incompatibilidades descritas en la introducción.

2.3 Estructura del repositorio en GitHub

```
readmision-api/
├── app.py                # API REST (backend)
├── entrenar_modelo.py    # entrenamiento offline -> model.pkl
├── simular.py            # simulación local
├── cliente_api.py        # cliente HTTP de prueba
├── requirements.txt       # dependencias con versiones fijadas
├── Dockerfile            # definición de la imagen
├── docker-compose.yml    # orquestación local
├── .dockerignore / .gitignore
├── templates/index.html  # frontend
├── tests/test_api.py     # pruebas funcionales (pytest)
├── evidencias/           # salidas de pruebas, tablero y evidencia Docker
└── README.md             # manual del repositorio
```

2.4 Construcción del contenedor (Dockerfile)

El Dockerfile define, paso a paso, cómo se construye la imagen. Las instrucciones principales son:

Instrucción	Función
FROM python:3.11-slim	Imagen base ligera compatible con el modelo
WORKDIR /app	Define el directorio de trabajo dentro del contenedor
COPY requirements.txt .	Copia primero las dependencias (aprovecha la caché de capas)

Instrucción	Función
RUN pip install --no-cache-dir	Instala las librerías sin caché para reducir el tamaño
COPY . .	Copia el código fuente del proyecto
RUN python entrenar_modelo.py	Entrena y genera model.pkl dentro de la imagen
EXPOSE 5000	Expone el puerto del servicio
HEALTHCHECK	Verifica periódicamente el endpoint /health
CMD gunicorn ... app:app	Comando de arranque del servidor de producción

2.5 Construcción y ejecución local del contenedor (comprobado)

Desde la raíz del proyecto se construye la imagen y se ejecuta el contenedor:

```
# Construir la imagen (el build entrena el modelo)
docker build -t readmision-api:1.0.0 .

# Ejecutar el contenedor
docker run -d -p 5000:5000 --name readmision-api readmision-api:1.0.0

# Verificar el estado del servicio
curl http://localhost:5000/health
# -> {"status":"ok","modelo_cargado":true,"version":"1.0.0"}
```

La construcción y ejecución se comprobaron con Docker 29.6.1. La imagen se generó correctamente y el contenedor quedó operativo con estado de salud healthy:

Verificación	Resultado obtenido
Imagen construida	readmision-api:1.0.0 — 699 MB (build exitoso)
Contenedor en ejecución	Up (healthy), puerto 0.0.0.0:5000 -> 5000/tcp
Healthcheck del contenedor	healthy
Servidor de aplicación	gunicorn 23.0.0, 2 workers escuchando en 0.0.0.0:5000

Alternativamente, con Docker Compose: docker compose up --build. El contenedor se inicia en segundos, sin arrancar un sistema operativo completo. La evidencia completa se incluye en el Documento de Validación y Pruebas y en la carpeta evidencias/ del repositorio.

2.6 Integración con API y frontend (endpoints)

Método	Ruta	Descripción
GET	/	Frontend (formulario web)
GET	/health	Estado del servicio y del modelo
GET	/metadata	Versión, algoritmo, umbral y variables
GET	/ejemplos	Casos de ejemplo precargados
POST	/predict	Predicción a partir de 12 variables clínicas

Ejemplo de consumo del endpoint de predicción:

```
curl -X POST http://localhost:5000/predict \
-H "Content-Type: application/json" \
-d '{"features": [90,15,22,24,9,350,205,48,10,8,0.1,11]}'

# Respuesta:
{
  "prediccion": 1,
  "etiqueta": "Readmitido (alto riesgo)",
  "probabilidad_readmision": 0.8922,
  "umbral_aplicado": 0.30
}
```

2.7 Estrategia de despliegue en la nube

El despliegue se realiza bajo un modelo de servicio PaaS (Plataforma como Servicio), en el cual el proveedor gestiona la infraestructura y el usuario se concentra en la aplicación containerizada. Este modelo ofrece el mejor equilibrio entre control y esfuerzo operativo para publicar la imagen Docker, habilitando escalabilidad automática y actualización continua.

Modelo de servicio	Rol en esta solución
IaaS	Alternativa de mayor control (VM propia con Docker); mayor esfuerzo operativo
PaaS (elegido)	Publicación directa de la imagen/contenedor con escalado gestionado
SaaS	No aplica: la solución es el servicio, no software de terceros
FaaS	Viable para inferencia por eventos; no ideal por la carga del modelo en memoria

Plataformas PaaS compatibles con contenedores: Google Cloud Run, Azure App Service (for Containers), AWS App Runner y Render. Todas parten de la misma imagen Docker, lo que preserva la portabilidad.

2.8 Procedimiento paso a paso (PaaS con contenedor)

- Paso 1. Publicar el repositorio en GitHub con el código fuente y el Dockerfile.
- Paso 2. Construir y etiquetar la imagen: `docker build -t readmision-api:1.0.0`.
- Paso 3. Autenticarse en el registro de contenedores del proveedor (por ejemplo, `gcloud auth / az acr login`).
- Paso 4. Subir la imagen al registro (`docker push <registro>/readmision-api:1.0.0`).
- Paso 5. Crear el servicio PaaS apuntando a la imagen, definir el puerto 5000 y las variables de entorno.
- Paso 6. Desplegar y obtener la URL pública HTTPS del servicio.
- Paso 7. Verificar el despliegue consultando `/health` y ejecutando `cliente_api.py` contra la URL pública.

Ejemplo con Google Cloud Run a partir del código fuente (build gestionado):

```
gcloud run deploy readmision-api \
--source . \
--port 5000 \
```

```
--region us-central1 \  
--allow-unauthenticated
```

2.9 Monitoreo, escalabilidad y rollback

- Monitoreo: el endpoint /health y el HEALTHCHECK del contenedor permiten al orquestador verificar la disponibilidad del servicio.
- Escalabilidad: el modelo PaaS escala automáticamente el número de instancias según la demanda; para orquestación avanzada se puede migrar a Kubernetes.
- Restauración rápida (rollback): al versionar las imágenes (1.0.0, 1.0.1, ...) es posible revertir a una versión anterior de forma inmediata, ideal para CI/CD.

2.10 Documentación técnica: ISO/IEC 23053 y Copilot

La documentación se estructura conforme a la norma ISO/IEC 23053 (marco del ciclo de vida de sistemas de IA basados en machine learning), que exige un propósito documentado, un proceso de diseño controlado, actividades de verificación y validación, supervisión operativa y una estrategia de fin de vida útil. La separación entre el entrenamiento offline (entrenar_modelo.py) y el uso del modelo (app.py) responde directamente a este marco.

Como acelerador de la redacción técnica se emplean herramientas de IA generativa (por ejemplo, Microsoft Copilot o Claude), que permiten generar plantillas, runbooks, listas de verificación y procedimientos reproducibles de forma rápida, estructurada y con nivel profesional, documentando no solo el entrenamiento sino también la operación, el monitoreo y el mantenimiento.

3. Conclusiones

La contenerización con Docker resuelve el problema central planteado: garantizar un comportamiento estable y reproducible del modelo al migrar entre entornos. Al empaquetar el modelo junto con su código, dependencias y entorno de ejecución en una imagen estandarizada, se eliminan las inconsistencias derivadas de diferencias de versiones y configuraciones.

La integración frontend-backend mediante una API REST habilita un consumo funcional, escalable y seguro del modelo, mientras que la estrategia de despliegue en la nube bajo un modelo PaaS aporta portabilidad, escalabilidad automática y capacidad de restauración rápida. En conjunto, el proyecto demuestra el tránsito completo de un modelo experimental hacia una solución operativa, documentada y alineada con estándares internacionales.

Referencias Bibliográficas

-
- Banerjee, I. (2024). Dockerizing Your Machine Learning Model: A Step-by-Step Guide. <https://medium.com/@indradumnabanerjee/dockerizing-your-machine-learning-model-a-step-by-step-guide-12e92f8c960e>
- China, C., y Goodwin, M. (s.f.). IaaS, PaaS, SaaS: ¿cuál es la diferencia? IBM. <https://www.ibm.com/mx-es/think/topics/iaas-paas-saas>
- Docker. (s.f.). The Industry-Leading Container Runtime. <https://www.docker.com/products/container-runtime/>

- Red Hat. (2026). Docker: qué es, cómo funciona y sus ventajas. <https://www.redhat.com/es/topics/containers/what-is-docker>
- SG Systems. (2025). ISO/IEC 23053 — Marco del ciclo de vida del sistema de IA. <https://sgsystemsglobal.com/es/glosario/>
- Susnjara, S., y Smalley, I. (s.f.). ¿Qué son los contenedores? IBM. <https://www.ibm.com/mx-es/think/topics/containers>
- The Postman Team. (2023). What are the components of an API? <https://blog.postman.com/what-are-the-components-of-an-api/>